



Arquitectura del sistema

Sistema: Tiendita Maday

Cliente: La Familia

Versión: 2.0

Fecha: 16 de julio de 2026

Revisión técnica: 3c24760 sobre main

Descripción actualizada de componentes, servicios, APIs, modelo de datos y flujo de información. La arquitectura corresponde al código existente, no a diagramas históricos.

1. Visión arquitectónica

Tiendita Maday usa una arquitectura web de tres capas con servicios externos:

- **Presentación:** aplicación Angular responsiva/PWA para los cinco roles.
- **Aplicación:** API REST en Node.js y Express, responsable de autorización y reglas de negocio.
- **Datos:** PostgreSQL para información transaccional, catálogos y auditoría.
- **Integraciones:** Google, PayPal, Cloudinary, correo, mapas y plataforma de despliegue.

Vista lógica del sistema desplegable en Railway

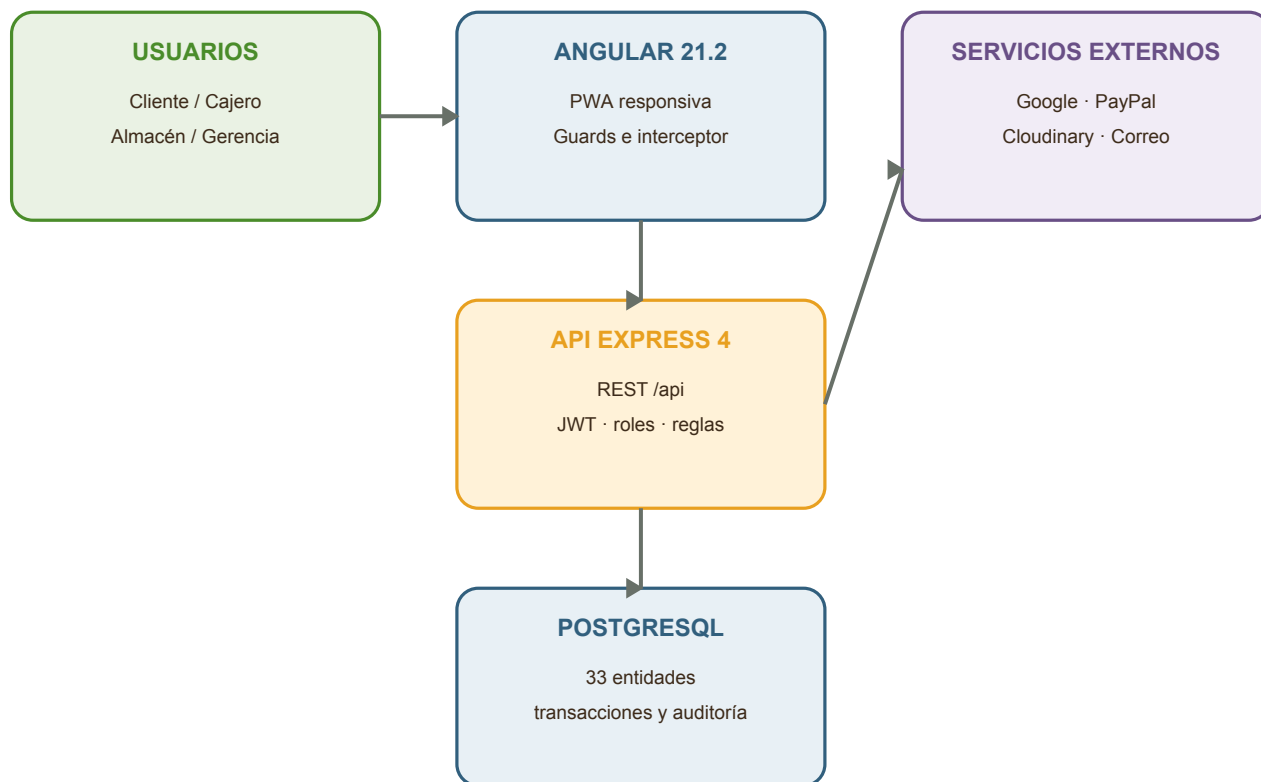


Diagrama arquitectónico actualizado a la revisión documentada.

El frontend nunca debe considerarse una frontera de seguridad. Aunque sus guards oculten rutas, la API vuelve a autenticar y autorizar cada operación protegida.

2. Principios de diseño

Principio	Aplicación en el proyecto
Servidor autoritativo	Precios, descuentos, envío, permisos y existencias se validan en la API.
Separación de responsabilidades	Componentes y servicios Angular; rutas, middleware, controladores, modelos y servicios backend.
Menor privilegio	Roles diferenciados y funciones de propietario reservadas al administrador exacto.
Trazabilidad	Sesiones, inventario, caja y cambios relevantes conservan evidencia.
Evolución de esquema	<code>init.sql</code> establece la base y las migraciones incrementales registran cambios.
Configuración por entorno	URLs y secretos se inyectan mediante variables; no se documentan valores sensibles.
Fallo visible	Errores centralizados y mensajes de interfaz sin confirmar operaciones incompletas.

3. Componentes del frontend

La aplicación utiliza Angular 21.2, TypeScript 5.9 y carga diferida. El enrutador principal divide el producto por dominio.

Área / ruta	Componentes principales	Responsabilidad
<code>/auth</code>	onboarding, login, registro, OTP, recuperación	Identidad, alta y recuperación.
<code>/home</code>	inicio, categorías, búsqueda, favoritos, notificaciones	Descubrimiento de productos.
<code>/product/:id</code>	detalle, galería, reseñas	Información y selección de producto.
<code>/cart</code>	carrito	Cantidades y subtotal preliminar.
<code>/checkout</code>	identificación, dirección, envío, pago, resumen	Captura guiada del pedido.
<code>/orders</code>	confirmación, error, historial, detalle	Seguimiento posventa.
<code>/profile</code>	datos, direcciones, pagos, ayuda, términos	Autoservicio del cliente.
<code>/cashier</code>	caja, venta, historial	Punto de venta y turno del cajero.
<code>/inventory</code>	existencias, movimientos	Control de almacén.
<code>/admin</code>	panel, productos, pedidos, usuarios, abasto, caducidades, promociones, finanzas, configuración	Gestión del negocio.

3.1 Servicios transversales del frontend

- Cliente HTTP y configuración de `API_BASE_URL`.
- Servicio de autenticación y almacenamiento controlado de sesión.
- Interceptor que agrega credenciales a solicitudes.
- Guards de autenticación, rol administrativo, cajero, almacén y propietario.

- Manejo de carrito y contexto del checkout.
- Integración con Leaflet/OpenStreetMap para ubicación y zonas.
- Generación de algunos comprobantes PDF con jsPDF.
- Service worker para capacidades PWA.

4. Componentes del backend

El backend sigue una separación por capas:

Capa	Ubicación	Función
Entrada	src/index.js	Express, CORS, JSON, OpenAPI, rutas y error global.
Rutas	src/routes/	URL, verbo HTTP y middleware requerido.
Middleware	src/middleware/	JWT, roles, cargas y manejo de errores.
Controladores	src/controllers/	Coordinación de cada solicitud y respuesta.
Modelos	src/models/	Consultas y persistencia PostgreSQL.
Servicios	src/services/	Integraciones y lógica reutilizable.
Utilidades	src/utls/	Validaciones y transformaciones aislables.
Configuración	src/config/	Base de datos, Cloudinary y entorno.

La API monta todos los recursos bajo /api, publica la especificación en /swagger.json y la interfaz Swagger UI en /api-docs.

5. Servicios de negocio

Dominio	Servicios y reglas principales
Autenticación	Registro, login, Google, refresh, logout, OTP, recuperación y perfil.
Catálogo	Categorías, unidades, productos, imágenes, precios, códigos y reseñas.
Inventario	Existencias, ajustes auditable, recepciones, lotes y caducidades.
Abastecimiento	Proveedores, órdenes de compra, recepción y devoluciones.
Ventas	Pedidos digitales, artículos, promociones, entrega y estados.
Punto de venta	Caja abierta, venta transaccional, efectivo/cambio, historial y corte.
Administración	Usuarios, empleados, horarios, clientes, notificaciones y configuración.
Finanzas	Gastos, movimientos y auditoría de caja para propietario.

6. Superficie de API

La especificación generada por src/openapi.js declara **82 rutas** y **138 operaciones HTTP** agrupadas en 28 etiquetas. Esta cifra describe la revisión documentada y debe recalcularse cuando cambie la API.

Grupo	Prefijo	Acceso típico
Autenticación	/api/auth	Público para login/registro; token para perfil y contraseña.
Configuración	/api/shop-config, /api/delivery-zones	Lectura controlada; cambios de propietario.
Catálogo	/api/categories, /api/units, /api/inventory, /api/reviews	Lectura pública/controlada; escritura por rol.
Abastecimiento	/api/suppliers, /api/purchase-orders, /api/stock-receipts	Administrador o gerente.
Personal	/api/employees, /api/schedules, /api/shift-covers	Administrador.
Clientes y usuarios	/api/users, /api/customers, /api/addresses, /api/payment-methods	Propietario de datos o personal autorizado.
Ventas	/api/purchases, /api/promotions, /api/paypal	Según operación y contexto autenticado.
Caja	/api/cash-register	Cajero o administrador.
Finanzas	/api/expenses, /api/till-movements, /api/cash-audit	Administrador exacto.

6.1 Contrato de solicitud y respuesta

- JSON es el formato principal.
- Los recursos usan identificadores UUID donde corresponde.
- El token se envía mediante el mecanismo de autorización configurado.
- Errores pasan por el middleware global y usan códigos HTTP coherentes.
- Las validaciones rechazan datos incompletos antes de escribir.
- La documentación OpenAPI es la referencia detallada para parámetros y respuestas.

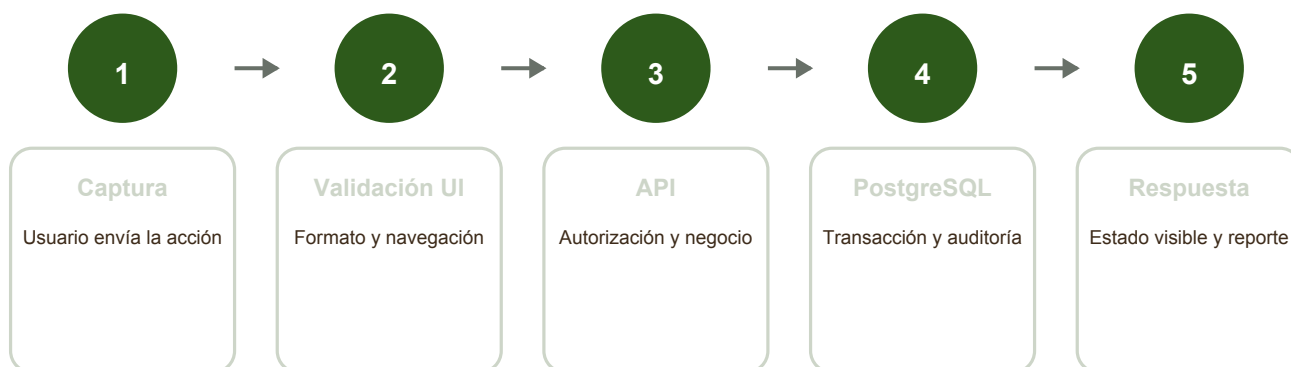
7. APIs y servicios externos

Servicio	Uso	Datos intercambiados	Degradación / control
Google Identity	Inicio de sesión opcional	Token de identidad y perfil mínimo	Mantener login local; validar cliente y origen.
PayPal REST	Pago digital	Orden, moneda, total e identificadores	Sandbox antes de live; evitar repetir un cobro.
Cloudinary	Imágenes de productos	Archivo, URL y public ID	Producto puede conservar imagen previa.
EmailJS / correo	OTP y recuperación	Destinatario, plantilla y código/enlace	Reintento controlado y soporte manual.
Leaflet + OpenStreetMap	Selección y visualización de ubicación	Coordenadas y teselas de mapa	La API calcula la tarifa; mapa no decide el total.
Railway	Despliegue administrado	Código, variables, logs y PostgreSQL	Respaldos exportables y registro de propietarios.

Las credenciales de terceros son secretos operativos. Se configuran fuera de Git y se transfieren por un canal seguro.

8. Flujo de información

Flujo principal de información



Los errores regresan por la misma ruta sin registrar una operación incompleta.

Secuencia de una operación de negocio desde la interfaz hasta la persistencia.

8.1 Flujo de autenticación

1. El usuario envía credenciales o token de Google.
2. La API valida identidad, estado de cuenta y contraseña/token.
3. El backend emite tokens con vigencia configurada y registra la sesión.
4. El frontend conserva el estado necesario y solicita el perfil.
5. Guards e interfaz dirigen al área del rol.
6. Cada endpoint protegido vuelve a verificar token y permisos.

8.2 Flujo de compra digital

1. El cliente consulta catálogo y arma un carrito local.
2. El checkout captura identidad, dirección, entrega y método de pago.
3. La API recupera productos vigentes y normaliza cantidades duplicadas.
4. El servidor calcula promociones, tarifa y total.
5. Si el pago requiere tercero, crea/captura la orden configurada.
6. PostgreSQL guarda compra y artículos y descuenta existencias de forma controlada.
7. La API devuelve folio y estado; el cliente consulta su historial.

8.3 Flujo de punto de venta

- 1. El cajero abre su caja con fondo y turno.
- 2. Busca productos y construye el ticket.
- 3. La API valida empleado, caja, productos y stock.
- 4. En una operación transaccional registra compra, artículos y descuento de inventario.
- 5. El sistema calcula cambio o registra la confirmación externa de tarjeta.
- 6. Al cierre compara efectivo contado contra esperado y conserva diferencia.

8.4 Flujo de ajuste de inventario

- 1. Almacén elige producto, entrada/salida, cantidad, motivo y nota.
- 2. La interfaz muestra una vista previa, pero la API recalcula.
- 3. El servidor rechaza cantidad inválida o stock negativo.
- 4. PostgreSQL actualiza existencia y crea el movimiento auditable.
- 5. La interfaz vuelve a consultar existencias e historial.

9. Modelo de datos

El archivo `Arquitectura/Database/schema.mmd` documenta **33 entidades** y sus relaciones. `init.sql` y las migraciones son las fuentes ejecutables.



Fuente canónica: `Arquitectura/Database/schema.mmd` + migraciones SQL

Mapa conceptual del modelo de datos; el detalle de relaciones permanece en `schema.mmd`.

9.1 Integridad y evolución

- Claves primarias UUID en la mayoría de entidades transaccionales.
- Llaves foráneas para identidad, catálogo, compras y auditoría.
- Restricciones y validaciones complementadas por la API.
- Transacciones en flujos que modifican varios registros.
- Tabla `schema_migrations` para no reaplicar archivos.
- Migraciones versionadas para usuarios, ventas, Google, roles, correo, inventario, administración, imágenes, checkout y reseñas.

10. Seguridad arquitectónica

Control	Implementación	Riesgo que reduce
Contraseñas	bcryptjs	Exposición directa de claves.
Sesiones	JWT de acceso/refresh y revocación	Uso indefinido de una sesión.
Autorización	<code>verifyToken</code> , <code>requireRole</code> , <code>requireExactRole</code>	Escalamiento de privilegios.
CORS	Origen de frontend configurable	Solicitudes desde sitios no previstos.
Validación	Controladores y utilidades del backend	Datos inválidos o manipulados por navegador.
Transacciones	Checkout, caja y configuraciones críticas	Escrituras parciales.
Secretos	Variables del entorno	Filtración en código o documentos.
Auditoría	Sesiones, login, inventario, precios, zonas y caja	Cambios sin responsable.

10.1 Fronteras de confianza

- El navegador y todo dato enviado por el usuario son no confiables.
- Los tokens son credenciales temporales y deben protegerse.
- La API confía en PostgreSQL solo mediante una conexión autenticada.
- Las respuestas de terceros deben verificarse antes de modificar pedidos.
- Los logs son evidencia operativa, no un lugar para secretos.

11. Despliegue físico

11.1 Desarrollo local

El repositorio ofrece proyectos separados de frontend, backend y base, además de configuración de contenedores. Para desarrollo directo se ejecutan dependencias en cada carpeta y PostgreSQL local o contenerizado.

11.2 Railway

Servicio	Entrada	Salida / dependencia
Frontend	Código Angular + API_BASE_URL de compilación	Sitio estático/PWA público.
Backend	Código Express + variables y secretos	API pública, conecta con PostgreSQL y terceros.
PostgreSQL	Volumen administrado y credenciales	Datos persistentes y migraciones.

El backend ejecuta migraciones antes de iniciar. El frontend debe apuntar a la API terminada en /api; el backend debe restringir CORS al dominio final.

12. Disponibilidad, respaldo y recuperación

- Frontend, backend y base pueden reiniciarse por separado.
- El código y las migraciones permiten reconstruir servicios.
- Los datos requieren respaldo lógico periódico con pg_dump.
- Una restauración se prueba primero en una base aislada.
- La copia fuera del proveedor reduce dependencia de Railway.
- La degradación de un tercero no debe confundirse con una venta confirmada.

Las instrucciones detalladas y los objetivos recomendados RPO/RTO se encuentran en los manuales técnicos.

13. Decisiones y pendientes arquitectónicos

Tema	Estado	Decisión / acción
Stack principal	Vigente	Angular 21.2 + Express 4 + PostgreSQL.
Documentación de API	Vigente	OpenAPI generada por src/openapi.js.
Fecha comercial	Hallazgo	Ajustar agrupación de ventas a zona horaria de tienda.
Tests integrales	Parcial	Ejecutar POS e inventario con bases aisladas.
Tests frontend	Pendiente	Alinear Vitest y archivos de especificación.
Marca	Pendiente de cliente	Normalizar "Tiendita Maday", "La Familia" y referencias históricas.
Accesibilidad	Parcial	Ejecutar auditoría WCAG y pruebas con teclado/lector.
Observabilidad	Básica	Definir métricas, alertas y retención de logs en producción.

14. Fuentes de verificación

Fuente	Qué verifica
FrontEnd/src/app/app.routes.ts y rutas de características	Módulos visibles, navegación y carga diferida.
Backend/src/index.js y src/routes/index.js	Montaje de API, CORS, middleware y políticas por dominio.
Backend/src/openapi.js	Contratos HTTP, etiquetas, parámetros y respuestas.
Database/schema.mmd	Entidades y relaciones lógicas.
Database/init.sql y migrations/	Esquema ejecutable y evolución de datos.
package.json de ambos proyectos	Frameworks, librerías y comandos soportados.
docker-compose.yml, Dockerfiles y Railway	Topología de ejecución y despliegue.

15. Lista de control al modificar la arquitectura

- [] Actualizar el diagrama si aparece o desaparece un componente.
- [] Actualizar OpenAPI si cambia una ruta, parámetro o respuesta.
- [] Agregar una migración idempotente si cambia el modelo de datos.
- [] Revisar autorización backend y guards frontend si cambia un rol.
- [] Documentar nuevas variables sin publicar sus valores.
- [] Agregar pruebas del flujo nuevo o modificado.
- [] Revisar respaldo, rollback y compatibilidad con la versión anterior.
- [] Actualizar documento general, implementación y manual técnico.

16. Control de versión arquitectónica

Campo	Valor
Rama revisada	main
Revisión base	3c24760
Fecha de análisis	16 de julio de 2026
Frontend	Angular 21.2 / TypeScript 5.9
Backend	Node.js / Express 4
Datos	PostgreSQL / 33 entidades documentadas
API	82 rutas / 138 operaciones documentadas

Si el código y un diagrama divergen, prevalecen las rutas, migraciones y pruebas de la revisión identificada; el diagrama debe actualizarse en el mismo cambio.